

Guide to Upgrade Payer Authentication in SAP Commerce B2C

Guide to Upgrade Payer Authentication in SAP Commerce B2C.....	1
Steps to Upgrade Cardinal Cruise Direct Connection API	2
1. Update the FlexMicroformController.java	2
2. Update SummaryCheckoutStepController.java	7

3. Update the GeneratedIsvPaymentConstants.java	7
4. Update the IsvPaymentConstants.java.....	7
5. Update the CreditCardRequestConverterHybrisModule.java	8
6. Update the EnrollmentRequestConverter.java	8
7. Update the CreditCardPaymentFacade.java	9
8. Update the CreditCardPaymentFacadeImpl.java	10
9. Add the SetUpRequestConverter.java file	12
10. Add the payerAuthHelper.jsp file	14
11. Update the flexCardPaymentDetails.js	14
12. Update the isvb2cpaymentaddon.js.....	15
13. Update the external-dependencies.xml	15
14. Update the isvpayment-items.xml	16
15. Update the isvpayment-integration-spring.xml	16
16. Update the isvpayment-converter-creditcard-spring.xml.....	17
17. Update isvpaymentaddon.css	17
18. Update cardinalCommerce.tag.....	18
19. Update the microform.tag	24
20. Update Configuration Properties (project.properties)	25

Steps to Upgrade Cardinal Cruise Direct Connection API

1. Update the FlexMicroformController.java

File Path:

hybris/bin/b2c/isvb2cpaymentaddon/acceleratoraddon/web/src/isv/sap/payment/addon/
b2c/controllers/pages/checkout/payment/flex/FlexMicroformController.java

Updated Code:

- **Add Import:**

```
import org.springframework.ui.Model;
```

- **Add Setup Method**

```
@ResponseBody
```

```
@PostMapping(path = "/attemptPaymentSetUp", produces =
MediaType.APPLICATION_JSON_VALUE)
public AjaxResponse setUp(
@RequestParam final String transientToken
)
{
    checkArgument(StringUtils.isNotBlank(transientToken), "transientToken is missing");
    String sanitizedTransientToken = StringEscapeUtils.escapeHtml4(transientToken);

    try
    {
        final IsvPaymentTransactionEntryModel setUpTransaction = creditCardPaymentFacade
            .setUpCreditCard(sanitizedTransientToken);
        final Map<String, String> properties = setUpTransaction.getProperties();
        if("ACCEPT".equals(properties.get("decision")))
        {
            return AjaxResponse.success()
                .put("decision", properties.get("decision"))
                .put("requestID", properties.get("requestID"))
                .put("deviceDataCollectionURL", properties.get("deviceDataCollectionURL"))
                .put("accessToken", properties.get("accessToken"))
                .put("referenceID", properties.get("referenceID"));
        }
        else
        {
            LOG.warn("Cart [{}]: Received invalid setup code [{}]. Payment should not proceed",
                cartService.getSessionCart().getCode(), properties.get("decision"));
        }
    }
    catch (final Exception ex)
```

```

{
LOG.error("Cart [{}]: Exception when trying to enroll/authorize credit card",
cartService.getSessionCart().getCode(), ex);
}
return AjaxResponse.fail()
.put("redirectUrl", URL_PAYMENT_FAILED);
}

```

- **Add PayerAuthHelper Method**

```

@PostMapping(path = "/payerAuthHelper")
public String payerAuthHelper(@RequestParam(name = "TransactionId") final String
transactionId, final Model model)
{
String sanitizedTransactionId = StringEscapeUtils.escapeHtml4(transactionId);
model.addAttribute("transactionId", sanitizedTransactionId);
return
"addon:/isvpaymentaddon/pages/checkout/multi/payment/payerAuthHelper";
}

```

- **Replace PayWithoutValidation Method**

```

@ResponseBody
@PostMapping(path = "/attemptPaymentWithoutValidation", produces =
MediaType.APPLICATION_JSON_VALUE)
public AjaxResponse payWithoutValidation(
@RequestParam final String referenceId,
@RequestParam final String transientToken,
@RequestParam final String browserCookieAccepted,
@RequestParam final String browserScreenHeight,
@RequestParam final String browserScreenWidth,
@RequestParam final String serviceReturnUrl,
final UriComponentsBuilder uriComponentsBuilder)
{
final String targetOrigin = uriComponentsBuilder
.replacePath(null).replaceQuery(null).userInfo(null).fragment(null)
.build()
.toUriString();
checkArgument(StringUtils.isNotBlank(referenceId), "referenceId is missing");
checkArgument(StringUtils.isNotBlank(transientToken), "transientToken is missing");
}

```

```

        checkArgument(StringUtils.isNotBlank(browserCookieAccepted), "User browser
cookie accepted is missing");
        checkArgument(StringUtils.isNotBlank(browserScreenHeight), "User browser screen
height is missing");
        checkArgument(StringUtils.isNotBlank(browserScreenWidth), "User browser screen
width is missing");
        String sanitizedTransientToken = StringEscapeUtils.escapeHtml4(transientToken);
        String sanitizedReferenceld = StringEscapeUtils.escapeHtml4(referenceld);
        String sanitizedBrowserCookieAccepted =
StringEscapeUtils.escapeHtml4(browserCookieAccepted);
        String sanitizedBrowserScreenHeight =
StringEscapeUtils.escapeHtml4(browserScreenHeight);
        String sanitizedBrowserScreenWidth =
StringEscapeUtils.escapeHtml4(browserScreenWidth);
        String sanitizedServiceReturnUrl = StringEscapeUtils.escapeHtml4(serviceReturnUrl);
        try
        {
            final IsvPaymentTransactionEntryModel enrollmentTransaction =
creditCardPaymentFacade
                .enrollCreditCard(sanitizedReferenceld, sanitizedTransientToken,
sanitizedBrowserCookieAccepted, sanitizedBrowserScreenHeight, sanitizedBrowserScree
nWidth, targetOrigin+sanitizedServiceReturnUrl+"/checkout/payment/flex/payerAuthHelpe
r");
            final Map<String, String> properties = enrollmentTransaction.getProperties();

            final String responseCode = properties.get("reasonCode");
            if (NOT_ENROLLED_CODE.equals(responseCode))
            {
                final String redirectUrl = payAndPlaceOrder(transientToken, null,
enrollmentTransaction);

                return AjaxResponse.success()
                    .put("responseCode", NOT_ENROLLED_CODE)
                    .put("redirectUrl", redirectUrl);
            }
            else if (ENROLLED_CODE.equals(responseCode))
            {
                return AjaxResponse.success()

```

```

        .put("responseCode", ENROLLED_CODE)
        .put("stepUpUrl", properties.get("payerAuthEnrollReplyStepUpUrl"))
        .put("accessToken", properties.get("payerAuthEnrollReplyAccessToken"))
        .put("paReq", properties.get("payerAuthEnrollReplyPaReq"));
    }
    else
    {
        LOG.warn("Cart [{}]: Received invalid enrollment code [{}]. Payment should not
proceed",
        cardService.getSessionCard().getCode(), responseCode);
    }
}
catch (final Exception ex)
{
    LOG.error("Cart [{}]: Exception when trying to enroll/authorize credit card",
    cardService.getSessionCard().getCode(), ex);
}
}

return AjaxResponse.fail()
    .put("redirectUrl", URL_PAYMENT_FAILED);
}

```

- **Changes in pay Method**

- a. Replace Method prototype

```

public AjaxResponse pay(@RequestParam(name = "transientToken") final String
transientToken,
    @RequestParam(name = "transactionId") final String transactionId)

```

- b. Replace this checkArgument(StringUtils.isNotBlank(authJwt), "authJwt is missing");
with

```

checkArgument(StringUtils.isNotBlank(transactionId), "transactionId is missing");

```

- c. Add argument sanitization for input fields after checkArguments

```

String sanitizedTransientToken = StringEscapeUtils.escapeHtml4(transientToken);
String sanitizedTransactionId = StringEscapeUtils.escapeHtml4(transactionId);

```

- d. Replace payAndPlaceOrder function Call in try block

```
final String redirectUrl = payAndPlaceOrder(sanitizedTransientToken,  
sanitizedTransactionId, null);
```

2. Update SummaryCheckoutStepController.java

File Path:

hybris/bin/b2c/isvb2cpaymentaddon/acceleratoraddon/web/src/isv/sap/payment/addon/
b2c/controllers/pages/checkout/steps/SummaryCheckoutStepController.java

Updated Code:

- Remove instance variable

```
@Value("${isv.payment.customer.3ds.songbird.url}")  
private String songbirdUrl;
```

- Changes in enterStep Method

- a. Remove `model.addAttribute("songbirdUrl", songbirdUrl);`
- b. Remove if block

```
if (is3dsEnabled && FLEX.equals(checkoutPciStrategy.getSubscriptionPciOption()))  
{  
    model.addAttribute("jwt", creditCardPaymentFacade.createEnrollmentJwt());  
}
```

3. Update the GeneratedIsvPaymentConstants.java

File Path:

/hybris/bin/b2c/isvpayment/gensrc/isv/sap/payment/constants/GeneratedIsvPaymentCo
nstants.java

Updated code:

- Add class variable in PaymentTransactionType class

```
public static final String SET_UP = "SET_UP".intern();
```

4. Update the IsvPaymentConstants.java

File Path:

/hybris/bin/b2c/isvpayment/src/isv/sap/payment/constants/IsvPaymentConstants.java

Updated Code:

- Add Class variables in CreditCardRequestFields class

```
public static final String HTTP_BROWSER_COOKIES_ACCEPTED =  
"billToHttpBrowserCookiesAccepted";  
public static final String HTTP_BROWSER_SCREEN_HEIGHT =  
"billToHttpBrowserScreenHeight";  
public static final String HTTP_BROWSER_SCREEN_WEIGHT =  
"billToHttpBrowserScreenWidth";  
public static final String ENROLL_SERVICE_RETURN_URL =  
"payerAuthEnrollServiceReturnURL";
```

5. Update the CreditCardRequestConverterHybrisModule.java

File Path:

/hybris/bin/b2c/isvpayment/src/isv/sap/payment/modules/converter/CreditCardRequest
ConverterHybrisModule.java

Updated Code:

- Add the import

```
import static isv.cjl.module.util.RequestConverterConstants.CreditCard.SET_UP;
```

- Add the instance variable to the Class

```
@Resource(name = "isv.sap.payment.converter.creditcard.setUpRequestConverter")  
private Converter<PaymentServiceRequest, Request> setUpRequestConverter;
```

- Add the setup service in Configure Method

```
bind(CONVERTER).annotatedWith(named(SET_UP)).toInstance(setUpRequestConverter);
```

6. Update the EnrollmentRequestConverter.java

File Path:

hybris/bin/b2c/isvpayment/src/isv/sap/payment/service/executor/request/converter/cred
itcard/EnrollmentRequestConverter.java

Updated Code:

- Replace the import statement

From: import static

isv.sap.payment.constants.IsvPaymentConstants.CreditCardRequestFields.CARD_IN
FO;

To:

import static

isv.sap.payment.constants.IsvPaymentConstants.CreditCardRequestFields.*;

- **Add the below lines in convert Method**

```
.addParam(HTTP_BROWSER_COOKIES_ACCEPTED,  
source.getRequiredParam("billToHttpBrowserCookiesAccepted"))  
.addParam(HTTP_BROWSER_SCREEN_HEIGHT,  
source.getRequiredParam("billToHttpBrowserScreenHeight"))  
.addParam(HTTP_BROWSER_SCREEN_WEIGHT,  
source.getRequiredParam("billToHttpBrowserScreenWidth"))  
.addParam(ENROLL_SERVICE_RETURN_URL,  
source.getRequiredParam("payerAuthEnrollServiceReturnURL"))
```

7. Update the CreditCardPaymentFacade.java

File Path:

/hybris/bin/b2c/isvpaymentaddon/src/isv/sap/payment/addon/facade/CreditCardPayment
Facade.java

Updated Code:

- **Add new Method prototype**

```
/**  
 * Does the setup request for the given payment information  
 *  
 * @param transientToken Flex microform token  
 * @return  
 */  
IsvPaymentTransactionEntryModel setUpCreditCard(final String transientToken);
```

- **Replace authorizeFlexCreditCardPayment Method Prototype**

```
boolean authorizeFlexCreditCardPayment(final CartModel cart, final String flexToken,
final String authJwt);
```

```
boolean authorizeFlexCreditCardPayment(final CartModel cart, final String flexToken,
final String transactionId);
```

- **Replace enrollCreditCard Method Prototype**

```
IsvPaymentTransactionEntryModel enrollCreditCard(final String referenceId, final
String transientToken);
```

```
IsvPaymentTransactionEntryModel enrollCreditCard(final String referenceId, final
String transientToken, final String browserCookiesAccepted, final String
browserScreenHeight, final String browserScreenWidth, final String serviceReturnURL);
```

- **Remove createEnrollmentJwt Method**

```
/**
 * Creates a new JWT (tokens must be regenerated even if the last one has not
expired)
 *
 * @return new JWT
 */
String createEnrollmentJwt();
```

8. Update the CreditCardPaymentFacadeImpl.java

File Path:

```
/hybris/bin/b2c/isvpaymentaddon/src/isv/sap/payment/addon/facade/CreditCardPaymen
tFacade.java
```

Updated Code:

- **Remove Imports**

```
import io.jsonwebtoken.Claims;
import isv.cjl.payment.service.jwt.JwtService;
```

- **Update import**

```
import static
isv.sap.payment.constants.IsvPaymentConstants.CreditCardRequestFields.FLEX_TOKEN
import static
isv.sap.payment.constants.IsvPaymentConstants.CreditCardRequestFields.*;
```

- **Remove Instance Variable**

```
@Resource(name = "isv.sap.payment.jwtService")
private JwtService jwtService;
```

- **Add setUpCreditCard Method**

```
@Override
public IsvPaymentTransactionEntryModel setUpCreditCard(final String
transientToken)
{
    final PaymentServiceResult setUpResult = executeRequest(
        new
isv.cjl.payment.service.executor.request.builder.creditcard.SetupRequestBuilder()
        .addParam(FLEX_TOKEN, transientToken)
        .setMerchantId(getMerchantID(CREDIT_CARD))
        .addParam(ORDER, cartService.getSessionCart())
        .build());
    return setUpResult.getData(TRANSACTION);
}
```

- **Changes in authorizeFlexCreditCardPayment Method**

- a. Replace Method prototype

```
public boolean authorizeFlexCreditCardPayment(final CartModel cart, final String
flexToken, final String authJwt)

public boolean authorizeFlexCreditCardPayment(final CartModel cart, final String
flexToken, final String transactionId)
```

- b. Remove Method variables decodedJwt, transactionId, errorNumber

```
final Claims decodedJwt =
jwtService.decodeJwt(authJwt, getSiteConfigService().getProperty("isv.payment.cus
tomer.3ds.jwt.api.key"));
```

```

final String transactionId = ofNullable(decodedJwt.get("Payload")).map(payload ->
((Map) payload).get("Payment")).map(payload -> (String) ((Map)
payload).get("ProcessorTransactionId")).orElse(null);
final Integer errorNumber = ofNullable(decodedJwt.get("Payload")).map(payload ->
(Integer) ((Map) payload).get("ErrorNumber")).orElse(null);

```

- c. Replace this condition `if (isNotBlank(transactionId) && Objects.equals(0, errorNumber))` with

```
if (isNotBlank(transactionId))
```

- **Add the parameters in EnrollCreditCard Method**

```

.addParam(HTTP_BROWSER_COOKIES_ACCEPTED, browserCookiesAccepted)
.addParam(HTTP_BROWSER_SCREEN_HEIGHT, browserScreenHeight)
.addParam(HTTP_BROWSER_SCREEN_WEIGHT, browserScreenWidth)
.addParam(ENROLL_SERVICE_RETURN_URL, serviceReturnURL)

```

- **Remove createEnrollmentJwt Method**

```

@Override
public String createEnrollmentJwt()
{
    final OrderData payload = enrollmentPayloadConverter
        .convert(cartService.getSessionCart());

    return jwtService
        .createEnrollmentJwt(getSiteConfigService().getProperty("isv.payment.custom
er.3ds.jwt.api.key"),
            payload);
}

```

9. Add the SetUpRequestConverter.java file

File Path:

/hybris/bin/b2c/isvpayment/src/isv/sap/payment/service/executor/request/converter/creditcard/SetUpRequestConverter.java

Updated Code:

```

package isv.sap.payment.service.executor.request.converter.creditcard;
import java.math.BigDecimal;
import de.hybris.platform.core.model.c2l.RegionModel;
import de.hybris.platform.core.model.order.AbstractOrderModel;
import de.hybris.platform.core.model.user.AddressModel;
import de.hybris.platform.payment.dto.CardInfo;
|
import isv.cjl.payment.configuration.transaction.PaymentTransaction;
import isv.cjl.payment.enums.CardType;
import isv.cjl.payment.exception.PaymentException;
import isv.cjl.payment.service.executor.request.PaymentServiceRequest;
import isv.cjl.payment.service.executor.request.converter.AbstractRequestConverter;
import isv.cjl.payment.service.request.Request;
import isv.cjl.payment.utils.Assert;
import isv.cjl.payment.utils.PaymentParamUtils;
|
import static isv.cjl.payment.constants.PaymentConstants.CommonFields.ORDER;
import static isv.cjl.payment.constants.PaymentRequestParamConstants.*;
import static isv.cjl.payment.constants.PaymentServiceConstants.CreditCard.SET_UP;
import static
isv.sap.payment.constants.IsvPaymentConstants.CreditCardRequestFields.CARD_INFO;
import static
isv.sap.payment.constants.IsvPaymentConstants.CreditCardRequestFields.FLEX_TOKEN
;
import static org.apache.commons.lang3.StringUtils.isEmpty;
|
/**
 * A component that encapsulates conversion logic from {@link PaymentServiceRequest}
 to
 * {@link Request} for payment setup request.
 */
public class SetUpRequestConverter extends AbstractRequestConverter
{
    @Override
    public Request convert(final PaymentServiceRequest source)
    {
        final AbstractOrderModel order = source.getRequiredParam(ORDER);
        final AddressModel billingAddress = order.getPaymentInfo().getBillingAddress();

```

```

    final AddressModel deliveryAddress = order.getDeliveryAddress();
    final String flexToken = source.getParam(FLEX_TOKEN);
    final PaymentTransaction setUpRequest = requestFactory.request(SET_UP)
        .addParam(MERCHANT_ID, source.getRequiredParam(MERCHANT_ID))
        .addParam(MERCHANT_REFERENCE_CODE, order.getGuid())
        .addParam(PAYER_AUTH_SET_UP_SERVICE_RUN, true)
        .addParam(TOKEN_SOURCE_TRANSIENT_TOKEN, flexToken);
    return setUpRequest.request();
}
}

```

10. Add the payerAuthHelper.jsp file

File Path:

hybris/bin/b2c/isvpaymentaddon/acceleratoraddon/web/webroot/WEB-INF/views/responsive/pages/checkout/multi/payment/payerAuthHelper.jsp

Updated Code:

```

<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ taglib prefix="template" tagdir="/WEB-INF/tags/responsive/template" %>
<html>
<head>
<template:javaScriptVariables/>
<script>
    window.parent.CARDINAL_COMMERCE.validation(window.parent.CARDINAL_COM
MERCERCE.transientToken,"${transactionId}");
</script>
</head>
</html>

```

11. Update the flexCardPaymentDetails.js

File Path:

/hybris/bin/b2c/isvpaymentaddon/acceleratoraddon/web/webroot/_ui/responsive/pages/js/checkout/multi/payment/flexCardPaymentDetails.js

Updated Code:

- **Changes in onTokenCreated Method:**

- a. Replace this if block

```
if (CARDINAL_COMMERCE.initialized) {  
    CARDINAL_COMMERCE.pay(transientToken)  
}  
if (CARDINAL_COMMERCE.setUp) {  
    CARDINAL_COMMERCE.showSpinner();  
    CARDINAL_COMMERCE.setUp(transientToken)  
}
```

12. Update the isvb2cpaymentaddon.js

File Path:

hybris/bin/b2c/isvb2cpaymentaddon/acceleratoraddon/web/webroot/_ui/responsive/common/js/isvb2cpaymentaddon.js

Updated Code:

- **Changes in submitFlexForm Method**

- a. Remove this if block

```
if (CARDINAL_COMMERCE.is3dsEnabled) {  
    CARDINAL_COMMERCE.initialize();  
}
```

13. Update the external-dependencies.xml

File Path:

hybris/bin/b2c/isvpayment/external-dependencies.xml.

Updated Code:

- **Remove the external dependencies**

```
<dependency>  
    <groupId>io.jsonwebtoken</groupId>  
    <artifactId>jjwt-api</artifactId>  
    <version>0.11.2</version>  
</dependency>  
<dependency>  
    <groupId>io.jsonwebtoken</groupId>
```

```

    <artifactId>jjwt-impl</artifactId>
    <version>0.11.2</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.2</version>
    <scope>runtime</scope>
  </dependency>

```

14. Update the isvpayment-items.xml

File Path:

/hybris/bin/b2c/isvpayment/resources/isvpayment-items.xml

Updated Code:

- **Add new value for enumtype code =PaymentTransactionType**

```

<value code="SET_UP">
  <description>set up service - checks that the customer in a card
  authentication program</description>
</value>

```

15. Update the isvpayment-integration-spring.xml

File Path:

hybris/bin/b2c/isvpayment/resources/payment/isvpayment-integration-spring.xml

Updated Code:

- **Remove Util constant of id JWT_SERVICE**

```

<util:constant id="JWT_SERVICE"
  static-field="isv.cjl.module.util.ConfigurationConstants.Jwt.JWT_SERVICE" />

```

- **Remove JWT bean with id isv.sap.payment.defaultJwtService**

```

<alias name="isv.sap.payment.defaultJwtService"
  alias="isv.sap.payment.jwtService"/>

```

```

<bean id="isv.sap.payment.defaultJwtService"
class="isv.sap.payment.injector.GuiceFactoryBean" lazy-init="true">
  <property name="beanClass">
    <value type="java.lang.Class">isv.cjl.payment.service.jwt.JwtService</value>
  </property>
  <property name="annotatedWith" ref="JWT_SERVICE" />
</bean>

```

16. Update the isvpayment-converter-creditcard-spring.xml

File Path:

hybris/bin/b2c/isvpayment/resources/payment/converter/isvpayment-converter-creditcard-spring.xml

Updated Code:

```

<alias name="isv.sap.payment.converter.creditcard.defaultSetUpRequestConverter"
alias="isv.sap.payment.converter.creditcard.setUpRequestConverter"/>

<bean id="isv.sap.payment.converter.creditcard.defaultSetUpRequestConverter"
class="isv.sap.payment.service.executor.request.converter.creditcard.SetUpRequestCon
verter"/>

```

17. Update isvpaymentaddon.css

File Path:

/hybris/bin/b2c/isvpaymentaddon/acceleratoraddon/web/webroot/_ui/responsive/comm
on/css/isvpaymentaddon.css

Updated Code:

```

#modal-container{
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  justify-content: center;
  align-items: center;
  z-index: 9999;
}

```

```

#modal-content{
padding: 20px;
border-radius: 5px;
text-align: center;
}
#step-up-iframe-id{
background-color: white;
}

```

18. Update cardinalCommerce.tag

File Path:

/hybris/bin/b2c/isvpaymentaddon/acceleratoraddon/web/webroot/WEB-INF/tags/responsive/payment/3ds/cardinalCommerce.tag

Updated Code:

- **Replace the content inside the file**

```

<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:choose>
  <c:when test="${is3dsEnabled && paymentPciType == 'FLEX'}">

```

```

    <script type="application/javascript">

```

```

      var CARDINAL_COMMERCE = {
        is3dsEnabled: true,
        initialized: false,
        referenceID: null,
        transientToken: null,
        flexToken: null,
        defaultErrorPage : '/checkout/multi/summary/view/payment/error',

```

```

        hideStepUpForm: function(){
          const modalContainerToNone = document.getElementById("modal-container");
          modalContainerToNone.style.display = "none";

```

```

    },
    validation: function(transientToken, transactionId){
        CARDINAL_COMMERCE.hideStepUpForm();
        CARDINAL_COMMERCE.showSpinner();
        //Validate payment and place order
        if(transientToken && transactionId) {
            $.ajax({
                url: ACC.config.contextPath + '/checkout/payment/flex/payWithValidation',
                cache: false,
                dataType: 'json',
                method: 'POST',
                data: {
                    transientToken: transientToken,
                    transactionId: transactionId
                },
                success: function (response) {
                    CARDINAL_COMMERCE.hideSpinner();
                    window.location.replace(ACC.config.contextPath +
response.data.redirectUrl);
                },
                error: function (response) {
                    CARDINAL_COMMERCE.hideSpinner();
                    console.error("There was an error while placing the order", arguments)
                    var redirectUrl =CARDINAL_COMMERCE.defaultErrorPage;
                    if (response.data && response.data.redirectUrl) {
                        redirectUrl = response.data.redirectUrl
                    }
                    window.location.replace(ACC.config.contextPath + redirectUrl);
                }
            });
        } else {
            window.location.replace(ACC.config.contextPath +
CARDINAL_COMMERCE.defaultErrorPage);
        }
    },
    setUp: function (transientToken) {

        CARDINAL_COMMERCE.transientToken=transientToken
    }
}

```

```
CARDINAL_COMMERCE.showSpinner();

$.ajax({
    url: ACC.config.contextPath +
'/checkout/payment/flex/attemptPaymentSetUp',
    cache: false,
    method: 'POST',
    data: {
        transientToken: transientToken
    },
    success: function (response) {
        if('ACCEPT'== response.data.decision){
            window.addEventListener("message", (event) => {
                if (event.origin === "https://centinelapistag.cardinalcommerce.com") {
                    let data = JSON.parse(event.data);
                    if (data != undefined && data.Status) {
                        console.log('Data received successfully');

```

```
CARDINAL_COMMERCE.pay(CARDINAL_COMMERCE.transientToken);
    }
    console.log('Merchant received a message:', data);
    }
    }, false);

    CARDINAL_COMMERCE.referenceID = response.data.referenceID;
    var formElement = document.createElement("form");
    var iframe = document.createElement('iframe');
    iframe.id = 'ddc_iframe';
    iframe.name = 'ddc_iframe';
    iframe.height = '10';
    iframe.width = '10';
    iframe.style.display = 'none';
    document.body.appendChild(iframe);

```

```
        formElement.id = "ddc_form";
        formElement.method = "post";
        formElement.target = "ddc_iframe";
        formElement.action = response.data.deviceDataCollectionURL;

```

```
        var inputElement = document.createElement("input");

```

```

        inputElement.id = "access_token";
        inputElement.type = "hidden";
        inputElement.name = "JWT";
        inputElement.value = response.data.accessToken;
        formElement.appendChild(inputElement);
        document.body.appendChild(formElement);
        var ddcForm = document.querySelector("#ddc_form");
        if (ddcForm) {
            ddcForm.submit();
        }
    }else if(response.data.redirectUrl){
        CARDINAL_COMMERCE.hideSpinner();
        window.location.replace(ACC.config.contextPath +
response.data.redirectUrl.redirectUrl);
    }else{
        CARDINAL_COMMERCE.hideSpinner();
        window.location.replace(ACC.config.contextPath + redirectUrl);
    }
},
    error: function (response) {
        CARDINAL_COMMERCE.hideSpinner();
        window.location.replace(ACC.config.contextPath + redirectUrl);
    }
});
},
    pay: function (transientToken) {
        CARDINAL_COMMERCE.showSpinner();
        $.ajax({
            url: ACC.config.contextPath +
'/checkout/payment/flex/attemptPaymentWithoutValidation',
            cache: false,
            method: 'POST',
            data: {
                transientToken: transientToken,
                referenceId: CARDINAL_COMMERCE.referenceId,
                browserCookieAccepted: navigator.cookieEnabled,
                browserScreenHeight: screen.height,

```

```
        browserScreenWidth: screen.width,
        serviceReturnUrl: ACC.config.contextPath
    },
    success: function (response) {
        if (response.data.responseCode === '475') {
            const challengeWindowSizeMap = {
                '01': { width: 250, height: 400 },
                '02': { width: 390, height: 400 },
                '03': { width: 500, height: 600 },
                '04': { width: 600, height: 400 },
                '05': { width: window.innerWidth, height: window.innerHeight },
                '06': { width: 400, height: 400 }
            };
            pareq = response.data.paReq;
            var decodedPareqValue = window.atob(pareq);
            var pareqJson = JSON.parse(decodedPareqValue);
            var challengeWindowSize = pareqJson.challengeWindowSize;
            const { width, height } = challengeWindowSizeMap[challengeWindowSize] ||
challengeWindowSizeMap['06'];

            var modalContainer = document.createElement('div');
            modalContainer.id = 'modal-container';
            modalContainer.style.display = 'none';

            var modalContent = document.createElement('div');
            modalContent.id = 'modal-content';

            var iframe = document.createElement('iframe');
            iframe.id = 'step-up-iframe-id';
            iframe.name = 'step-up-iframe';

            var form = document.createElement('form');
            form.id = 'step-up-form';
            form.target = 'step-up-iframe';
            form.method = 'post';

            var accessTokenInput = document.createElement('input');
            accessTokenInput.type = 'hidden';
```

```

        accessTokenInput.id = 'accessToken';
        accessTokenInput.name = 'JWT';

        var merchantDataInput = document.createElement('input');
        merchantDataInput.type = 'hidden';
        merchantDataInput.id = 'merchantData';
        merchantDataInput.name = 'MD';

        form.appendChild(accessTokenInput);
        form.appendChild(merchantDataInput);
        modalContent.appendChild(iframe);
        modalContent.appendChild(form);
        modalContainer.appendChild(modalContent);
        document.body.appendChild(modalContainer);

        $('#step-up-iframe-id').attr({width, height});
        $("#step-up-form").attr("action", response.data.stepUpUrl);
        $("#accessToken").val(response.data.accessToken);
        var stepupForm = document.querySelector("#step-up-form");
        if (stepupForm) {
            const modalContainerConst = document.getElementById("modal-
container");
            modalContainerConst.style.display = "block";
            CARDINAL_COMMERCE.hideSpinner();
            stepupForm.submit();
        }
        } else {
            //Card not enrolled. Payment was authorized and we can redirect to order
confirmation
            window.location.replace(ACC.config.contextPath +
response.data.redirectUrl);
        }
    },
    error: function (response) {
        CARDINAL_COMMERCE.hideSpinner();
        console.error("Error enrolling card", arguments)

        var redirectUrl = CARDINAL_COMMERCE.defaultErrorPage;

```

```

        if (response.data && response.data.redirectUrl) {
            redirectUrl = response.data.redirectUrl
        }

        window.location.replace(ACC.config.contextPath + redirectUrl);
    }
});
},

showSpinner: function() {
    $('.spinner-wrapper').show();
    $('#placeOrder').prop('disabled', true)
},

hideSpinner: function() {
    $('.spinner-wrapper').hide();
    $('#placeOrder').prop('disabled', false)
}
}

</script>
</c:when>
<c:otherwise>
    <script type="application/javascript">
        var CARDINAL_COMMERCE = {
            is3dsEnabled: false
        }
    </script>
</c:otherwise>
</c:choose>

```

19. Update the microform.tag

File Path:

hybris/bin/b2c/isvpaymentaddon/acceleratoraddon/web/webroot/WEB-INF/tags/responsive/payment/flex/microform.tag

Updated Code:

- Changes in createNewJwk Method
 - a. Replace this microform variable

```
var microform = flex.microform({styles: MICROFORM.onGetStyles()});
```

```
var microform = flex.microform('card', {styles: MICROFORM.onGetStyles()});
```

20. Update Configuration Properties (project.properties)

File Path:

hybris/bin/b2c/isvpayment/project.properties

Updated properties:

- Updated WSDL version

```
# SOAP WS URL
```

```
isv.payment.ws.endpoint.address=https\://ics2wstesta.ic3.com/commerce/1.x/transactionProcessor/CyberSourceTransaction_1.219.wsdl
```

```
# If a new wsdl location is required to override default supplied
```

```
isv.payment.ws.wsdl.location=classpath\META-INF/wsdl/CyberSourceTransaction_1.219.wsdl
```

- Remove 3ds related properties

```
# 3D Secure
```

```
isv.payment.customer.3ds.songbird.url=https\://songbirdstag.cardinalcommerce.com/edge/v1/songbird.js
```

```
isv.payment.customer.3ds.jwt.api.key=<3ds Api Key>
```

```
isv.payment.customer.3ds.jwt.api.orgUnitId=<3ds orgUnit ID>
```

```
isv.payment.customer.3ds.jwt.api.id=<3ds api ID>
```

```
#isv.payment.customer.3ds.<merchantID>.enabled=true
```